

14:35

Wednesday, October 16

26°C Good 27°C 75%

第二章：算法的性能分析

算法设计与分析



主讲教师：胡一涛

目录大纲

□ 算法分析概述

- 定义与分类
- 问题与问题实例
- 算法分析的原则

□ 空间复杂度

□ 时间复杂度



常用的分析方法

- 操作计数
- 程序步计数

- 最好、最坏与平均情况



★ 渐进分析法



★ 考察重点



熟练掌握技巧

目录大纲

□ 算法分析概述

- 定义与分类
- 问题与问题实例
- 算法分析的原则

□ 空间复杂度

□ 时间复杂度



常用的分析方法

- 操作计数
- 程序步计数

- 最好、最坏与平均情况



渐进分析法



考察重点



熟练掌握技巧

2.1 什么是算法分析?

参见课本: P030-P031

算法分析是指对算法的执行时间和所需空间的估算。当算法写成程序后, 可以通过对**程序的性能测量**来分析算法。

□ 什么是程序的性能?

- 指程序运行时所需的**内存空间量**和**计算时间**: 系统开销和求解问题本身的开销。
- 如何忽略系统开销, 区分出算法的性能?

2.1 什么是算法分析？

参见课本：P030-P031

□ 为什么要研究程序的性能？

- 设计满足要求的算法

□ 性能评价的方法？

- **解析**的方法
- 测量的方法

本课程以解析方法为主——算法分析

2.1 什么是算法分析?

参见课本: P030-P031

算法性能分析是评价算法优劣的重要依据

时间和空间 (即内存) 是计算机最重要的两种资源, 可以用来度量算法的性能:

■ **时间复杂性**: 算法需要时间资源的量。

$$T(n)$$

Q: n 指的是什么?

■ **空间复杂性**: 算法需要的空间资源的量。

$$S(n)$$

目录大纲

□ 算法分析概述

- 定义与分类
- 问题与问题实例
- 算法分析的原则

□ 空间复杂度

□ 时间复杂度



常用的分析方法

- 操作计数
- 程序步计数

- 最好、最坏与平均情况



渐进分析法



考察重点



熟练掌握技巧

什么是问题？

参见课本：P037

问题：需要人们回答的一般性提问，通常由**问题描述**、**输入条件**、**输出**要求等要素组成

- 排序问题：任意给定具有“序”的关系的集合 U 上的 n 个元素，试将这些元素按“从小到大”进行排列



什么是问题实例？

参见课本：P037

问题实例： 确定问题描述参数后的一个对象

- 例如：任给 $n = 100$ 个整数，试将其排序
- 问题实例的描述参数 n 称为问题的**实例特征**



算法分析建立算法复杂性和实例特征的函数关系

目录大纲

□ 算法分析概述

- 定义与分类
- 问题与问题实例
- 算法分析的原则

□ 空间复杂度

□ 时间复杂度



常用的分析方法

- 操作计数
- 程序步计数

- 最好、最坏与平均情况



渐进分析法



考察重点



熟练掌握技巧

算法分析的原则

□ 机器的运算速度影响算法的运行时间

机器	运算速度	运行时间
天河三号	百亿亿次/秒	无法公平比较
个人电脑	十亿次/秒	



VS
?



分析算法的运行时间应独立于机器

算法分析的原则

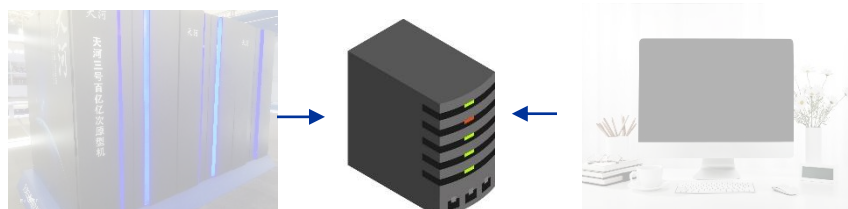
□ 归纳基本操作

- 如：运算、赋值、比较

+	-	×	÷
:=	>	<	=

□ 统一机器性能

- 假设基本操作代价均为1



算法复杂性依赖于：

问题的规模、算法的输入和算法本身的函数

目录大纲

□ 算法分析概述

- 定义与分类
- 问题与问题实例
- 算法分析的原则

□ 空间复杂度

□ 时间复杂度



常用的分析方法

- 操作计数
- 程序步计数

- 最好、最坏与平均情况



渐进分析法



考察重点



熟练掌握技巧

2.2 空间复杂性

参见课本：P031-P036

□ 算法的**空间复杂度**指算法执行时所需的存储空间，是判断算法优劣的重要度量指标

空间复杂度分析的意义

- 判断是否有足够的内存
- 在多用户系统中指明算法所需内存的大小
- 在多个可选择算法中“择优”
- 估算算法所能解决的问题规模上限

2.2.1 空间复杂性的组成

参见课本：P031-P033

概念

程序p的**空间复杂度**指程序运行时所需的内存空间大小和实例特征的函数关系

程序运行时所需空间包括:

- **指令空间**: 编译之后的程序指令所需要的存储空间
- **数据空间**:
 - 常量和简单变量所需要的存储空间
 - 复合变量(数组、链表、树和图等)所需空间
- **环境栈空间**: 用来保存函数调用返回时恢复运行所需要的信息

注: 复合变量所需空间常常和问题实例特征有关

2.2.1 空间复杂性的组成

参见课本：P034

- 程序 p 的空间需求量包括两部分：

$$S(n) = c + S_p(n)$$

c 为常量(实例无关部分)

$S_p(n)$ 为可变部分

- 在使用解析方法研究程序 p 的空间复杂度仅考虑函数 $S_p(\text{instance characteristics})$
- 在分析空间复杂度时我们**忽略与实例特征无关的空间需求量**

2.2.2 空间复杂性的实例分析

参见课本：P035

□ **例2-2 顺序查找：**在无序列表a[]中查找x，若找到，返回其位置；否则返回-1

```
Template <class T>
```

```
int SequentialSearch(T a[], const T& x, int n)
```

```
{int i;
```

```
    for (i = 0; i < n && a[i] != x; i++);
```

```
    if (i == n) return -1;
```

```
    return i;
```

```
}
```

T a[]和T& x需2bytes
指针(假定T为整形)

形参n需2bytes

i需2bytes

以上均为实例特征独立的空间需求量， $S_p(n)=0$

2.2.2 空间复杂性的实例分析

参见课本：P036

□ 例2-4 递归求和：返回列表a[0:n-1]的数字总和

```
Template <class T>
T RSum(T a[], int n)
{
    if (n > 0)
        return Rsum(a, n-1) + a[n-1];
    return 0;
}
```

核心问题

- 计算每次递归所占空间
- 确定递归深度

2.2.2 空间复杂性的实例分析

参见课本：P036

□ 例2-4 递归求和：返回列表a[0:n-1]的数字总和

```
Template <class T>
```

```
T RSum(T a[], int n)
```

```
{
```

```
    if (n > 0)
```

```
        return Rsum(a, n-1) + a[n-1];
```

```
    return 0;
```

```
}
```

T a[] 需2bytes指针
形参n需要2bytes

n=0时

形参n需2bytes

2.2.2 空间复杂性的实例分析

参见课本：P036

□ 例2-4 递归求和：返回列表a[0:n-1]的数字总和

```
Template <class T>
T RSum(T a[], int n)
{
    if (n > 0)
        return Rsum(a, n-1) + a[n-1];
    return 0;
}
```

T a[] 需2bytes指针
形参n需要2bytes

n=1时

返回地址需2bytes

递归栈需 6 bytes,
 $SP(n) = 6(n+1)$ bytes

注：递归深度是 $n+1$ 。

2.2.2 空间复杂性的实例分析

参见课本：P036

□ 例2-5 计算 $n!$ (n 的阶乘)

```
int Factorial(int n)
```

形参 n 需要2bytes

```
{
```

```
    if ( $n \leq 1$ ) return 1;
```

```
    else return  $n * \text{Factorial}(n - 1)$ ;
```

递归深度?

```
}
```

返回地址需2bytes

注：递归深度是 $\max\{n, 1\}$ 。

递归栈需 4 bytes,

$SP(n) = 4 \times \max\{n, 1\}$ bytes

2.2 空间复杂性的总结

对非递归算法：

分析与实例特征有关的**数据结构的大小**

对递归算法：

还要分析**递归调用的深度和实例特征的关系**，而且这往往是决定递归栈空间大小的主要因素

注：随着半导体技术的飞跃式发展，存储器资源的成本越来越低，人们对于算法空间复杂性的关注度也越来越低

本课程主要分析算法的时间复杂度

目录大纲

□ 算法分析概述

- 定义与分类
- 问题与问题实例
- 算法分析的原则

□ 空间复杂度

□ 时间复杂度



常用的分析方法

- 操作计数
- 程序步计数

- 最好、最坏与平均情况



渐进分析法



考察重点



熟练掌握技巧

2.3 时间复杂度分析

参见课本：P037-P074

时间复杂度指程序执行时所用的时间

□ 在解析地分析时间复杂度时,使用以下两种时间单位并计算:

- **操作计数**(operation count):找出一个或多个关键操作,确定这些关键操作所需要的执行时间
- (程序)**步计数**(step count):确定程序总的执行步数
- **要点**:基本操作或程序步的执行时间必须是常数



目录大纲

□ 算法分析概述

- 定义与分类
- 问题与问题实例
- 算法分析的原则

□ 空间复杂度

□ 时间复杂度



常用的分析方法

- 操作计数

- 程序步计数

- 最好、最坏与平均情况



渐进分析法



考察重点



熟练掌握技巧

2.3.2 时间复杂度分析

参见课本：P037

时间复杂度指程序执行时所用的时间

□ 在解析地分析时间复杂度时,使用以下两种时间单位并计算:

- **操作计数(operation count)**:找出一个或多个关键操作,确定这些关键操作所需要的执行时间

- (程序)步计数(step count):确定程序总的执行步数

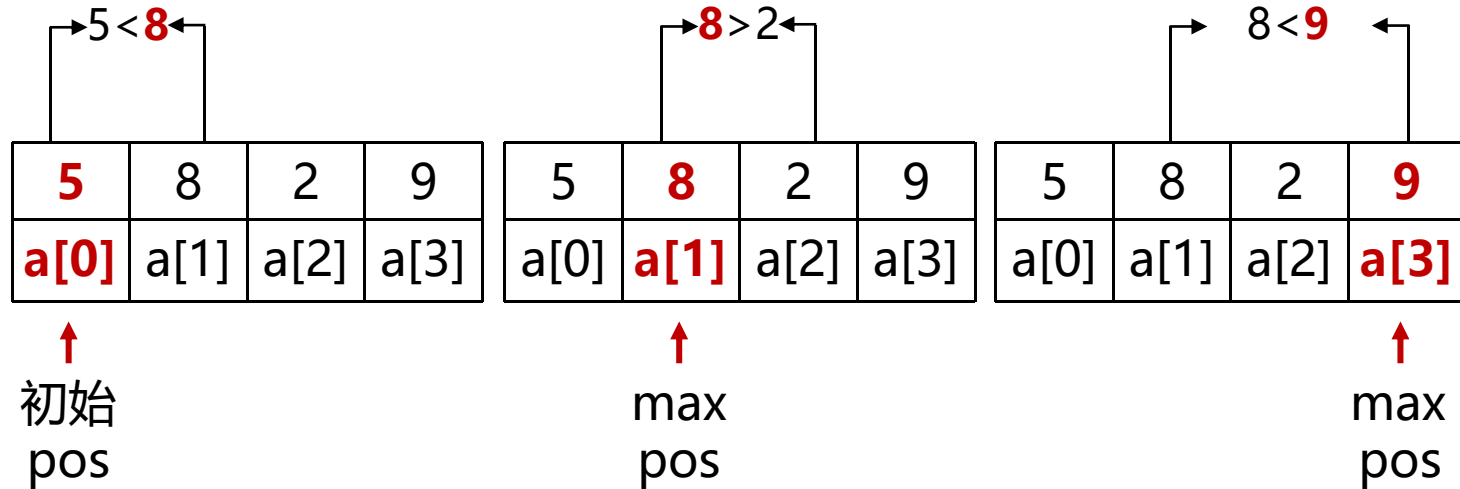
- **要点：基本操作或程序步的执行时间必须是常数**



2.3.2 操作计数（最大元素）

参见课本：P037

□ 例2-7 最大元素：从数组a [0,n-1]中找出最大元素的位置。可以根据数组元素之间**进行比较的次数**来估算时间复杂性



2.3.2 操作计数（最大元素）

参见课本：P037

```
template <class T>
int Max(T a[], int n)
{ // Locate the largest element in a[0:n - 1].
    int pos = 0;
    for (int i = 1; i < n; i++)
        if (a[pos] < a[i])
            pos = i;
    return pos;
}
```

□ 实例特征： n

□ 选定的操作：

数组元素间的比较次数

□ $T = n - 1$

2.3.2 操作计数（多项式求值）

参见课本：P037

□ 例2-8 多项式求值：对于 $c_n \neq 0$ 的 n 维多项式 $P(x) = \sum_{i=0}^n c_i x^i$ ，给定 x 求 $P(x)$

- 可以根据for循环内部所执行的**加和乘的次数**来估算时间复杂度

$$P(x) = \sum_{i=0}^n c_i x^i = c_0 + c_1 x + c_2 x^2 + \dots + c_n x^n$$

2.3.2 操作计数（多项式求值）

参见课本：P038

$$P(X) = \sum_{i=0}^n c_i X^i = c_0 + c_1 X + c_2 X^2 + \dots + c_n X^n$$

```
template <class T>
```

```
T PolyEval (T coeff[], int n, const T& x)
```

```
{// Evaluate the degree n polynomial with coefficients
```

```
// coeff[0:n] at the point x.
```

```
    T y = 1, value = coeff[0];
```

```
    for (int i = 1; i <= n; i++)
```

```
    {// add in next term
```

```
        y *= x;
```

```
        value += y * coeff[i];
```

```
    }
```

```
    return value;
```

```
}
```

□ 实例特征：n

□ 选定的操作：乘法次数

□ for循环内的乘法次数是2

□ for循环的深度为n

□ $T1 = 2n$

2.3.2 操作计数（秦九韶-霍纳算法）

参见课本：P038

□ 例2-8 多项式求值：对于 $c_n \neq 0$ 的 n 维多项式 $P(x) = \sum_{i=0}^n c_i x^i$ ，给定 x 求 $P(x)$

- 可以根据for循环内部所执行的**加和乘的次数**来估算时间复杂度

$$P(x) = \sum_{i=0}^n c_i x^i = (\dots ((c_n * x + c_{n-1}) * x + c_{n-2}) * x + c_{n-3}) * x \dots) * x + c_0$$

2.3.2 操作计数（秦九韶-霍纳算法）

参见课本：P038

```
template <class T>
T Horner(T coeff[], int n, const T& x)
{
    // Evaluate the degree n polynomial with coefficients
    // coeff[0:n] at the point x.
    T value = coeff[n];
    for (int i = 1; i <= n; i++)
        value = value * x + coeff[n - i];
    return value;
}
```

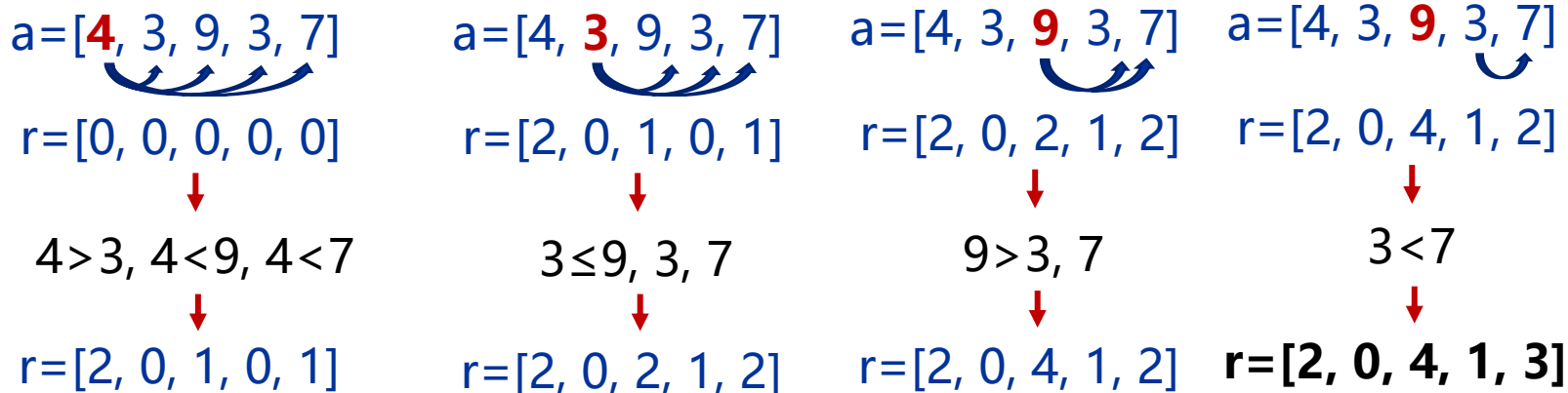
- ❑ 选定的操作：乘法次数
- ❑ for循环内乘法次数 1
- ❑ for循环的深度为n
- ❑ $T2 = n < T1$
- ❑ 说明该方法更快！

2.3.2 操作计数 (计算名次)

参见课本: P038

□ **例2-9 计算名次:** 元素在队列中的名次(rank)可定义为元素按大小顺序排列的位置。例如, 给定数组 $a=[4, 3, 9, 3, 7]$ 作为队列, 则各元素的名次 $r=[2, 0, 4, 1, 3]$ (由小到大)。

将数组a中的每个元素依次与后面的进行比较, 给更大的数名次 $r + 1$



2.3.2 操作计数（计算名次）

参见课本：P038

```
template < class T>
void Rank (T a[], int n, int r[])
{
    // Rank the n elements a[0:n-1].
    for (int i = 0; i < n; i++)
        r[i] = 0; //initialize
    // compare all element pairs
    for (int i = 1; i < n; i++)
        for (int j = 0; j < i; j++)
            if (a[j] <= a[i]) r[i]++;
            else r[j]++;
}
```

- 实例特征：n
- 选定的操作：数组元素的比较次数
- 循环内的比较次数是1
- 循环的深度是 $\sum i$
- $T = n(n-1)/2$

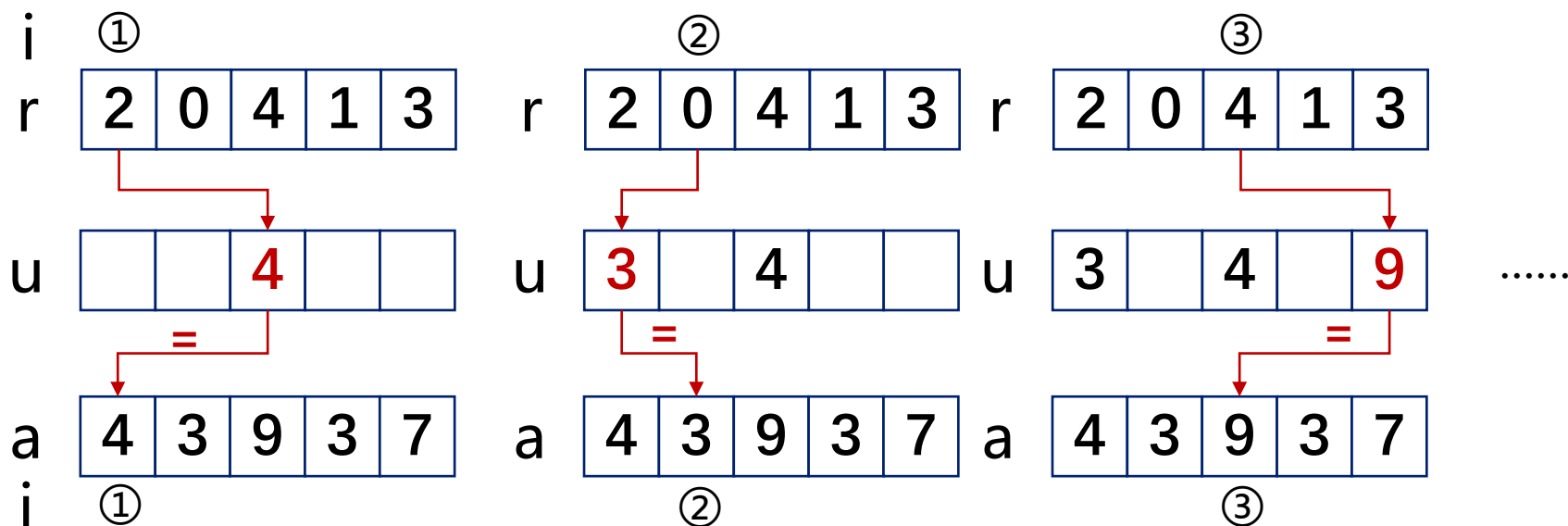
2.3.2 操作计数 (计数排序)

参见课本: P039

□ **例2-10 计数排序:** 在上一个程序中, 我们得到了数组 $a=[4, 3, 9, 3, 7]$ 的名次 $r=[2, 0, 4, 1, 3]$, 则可以根据 r 对 a 内的元素按名次排序, 使得 $a[0] \leq a[1] \leq \dots \leq a[n-1]$ 。

- 可以用一个附加数组 u 来帮助排序。

把 r 当作索引数组, 从 a 中选出对应的元素填入 u ; r 和 a 的遍历顺序是一致的



2.3.2 操作计数 (计数排序)

参见课本: P039

```
template <class T>
void Rearrange (T a[], int n, int r[])
{
    // Rearrange the elements of a into sorted order using
    // an additional array u.
    T* u = new T [n+1];
    // move to correct place in u
    for (int i = 0; i < n; i++)
        u[r[i]] = a[i];
    // move back to a
    for (int i = 0; i < n; i++)
        a[i] = u[i];
    delete [] u;
}
```

调用名次计算
 $r[i] = \text{rank of } a[i]$

- 实例特征: n
- 选定的操作: 元素移动
次数(即赋值)
- 循环内的赋值次数是1
- 循环的个数是2
- 每个循环的深度是 n
- $T = 2n$

2.3.2 操作计数 (选择排序)

参见课本: P039

- 例2-11 选择排序: 另外一种方法是先找出**最大元素**, 把它移动到 $a[n-1]$, 然后在余下的 $n-1$ 个元素中寻找最大的元素并把它移动到 $a[n-2]$, 如此进行下去, 这种排序方法为选择排序 (selection sort)



2.3.2 操作计数（选择排序）

参见课本：P039

```
template <class T>
void SelectionSort (T a[], int n)
{ // Sorted the n elements a[0:n-1].
    for (int size = n; size > 1; size-- ) {
        int j = Max(a, size);
        Swap(a[j], a[size - 1]);
    }
}
```

调用最大元素Max程序比较次数 $size - 1$

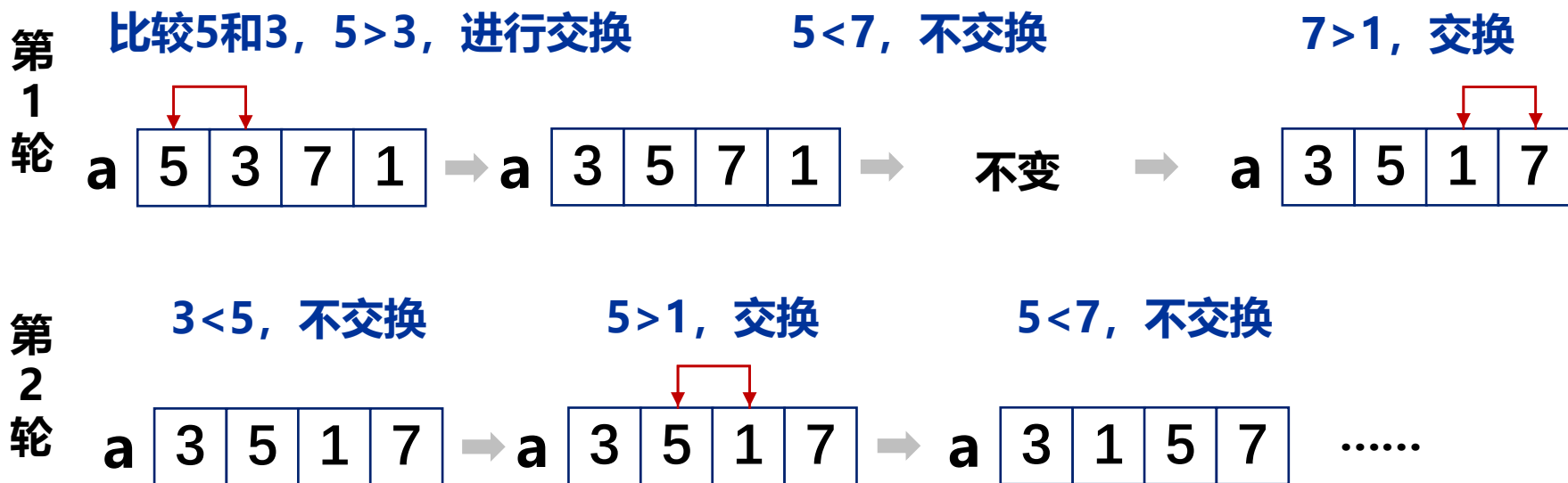
注：Swap原理
temp = y;
y = x;
x = temp;

- 实例特征为 n
- 选定的操作是比较和赋值
- Max的比较次数是 $size - 1$
- $T_{\text{比较}} = \sum (size - 1) = n(n - 1) / 2$
- Swap操作需3次简单赋值
- $T_{\text{赋值}} = 3(n - 1)$

2.3.2 操作计数（冒泡排序）

参见课本：P040

□ **例2-12 冒泡排序：**冒泡排序是另一种简单的排序方法。在冒泡过程中，对两两相邻的元素进行比较，如果左边的元素大于右边的，则交换两个元素，将最大的元素逐步移到最右侧。每执行1次冒泡过程，元素比较的次数为 $n-1$



2.3.2 操作计数（冒泡排序）

参见课本：P040

```
template <class T>
void bubble(T a[], int n)
{ // Move the largest element of a[0:n-1] to right.
    for (int i = 0; i < n-1; i++)
        if (a[i] > a[i+1])
            Swap(a[i], a[i+1]);}

template <class T>
void bubbleSort(T a[], int n)
{ // Sorted a[0:n-1]
    for (int i = n; i > 1; i--)
        bubble(a, i);
}
```

- 实例特征：n
- 选定的操作：比较次数。
- 一次冒泡的比较次数是n-1
- 排序中循环的深度是n
- $T = \sum(n-1) = (n-1)n/2$

目录大纲

□ 算法分析概述

- 定义与分类
- 问题与问题实例
- 算法分析的原则

□ 空间复杂度

□ 时间复杂度



常用的分析方法

- 操作计数
- 程序步计数

- 最好、最坏与平均情况



渐进分析法



考察重点



熟练掌握技巧

□ 时间复杂度与**问题的规模**有关

- 例如分别对数组A和B排序

数组A

5	8	2	9	3
---	---	---	---	---

VS

数组B

6	3	11	7	14	8	5	15	1	2	4	13	9	10	12
---	---	----	---	----	---	---	----	---	---	---	----	---	----	----

哪种排序耗费时间更短？

□ 时间复杂度与**输入**有关

- 例如分别对数组A和B从小到大排序

数组A

1	2	3	4	5	6	7	8	9	10
---	---	---	---	---	---	---	---	---	----

VS

数组B

7	1	2	6	10	4	8	3	9	5
---	---	---	---	----	---	---	---	---	---

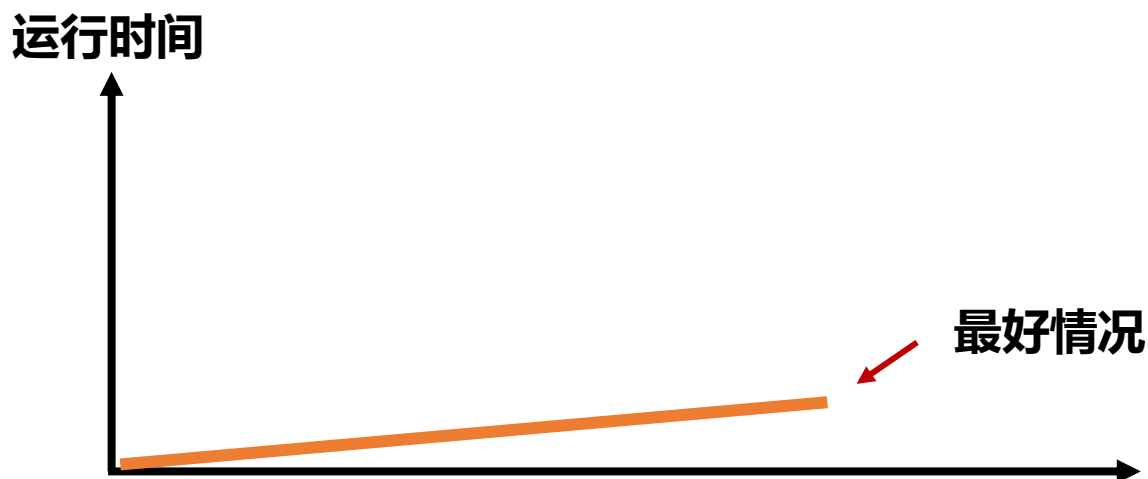
哪种排序耗费时间更短?

2.3.2 操作计数（最好、最坏与平均情况）

参见课本：P040

□ 算法分析分最好、最坏和平均情况

输入情况	情况说明
最好情况	不常出现，不具有普遍性



2.3.2 操作计数（最好、最坏与平均情况）

参见课本：P040

□ 算法分析分最好、最坏和平均情况

输入情况	情况说明
最好情况	不常出现，不具有普遍性

- 2008 年 5 月 15 日，捷克魔术师兹德涅克·布拉德克仅用时 36.16 秒就把 52 张扑克牌排好了先后次序。



最好成绩不靠谱！

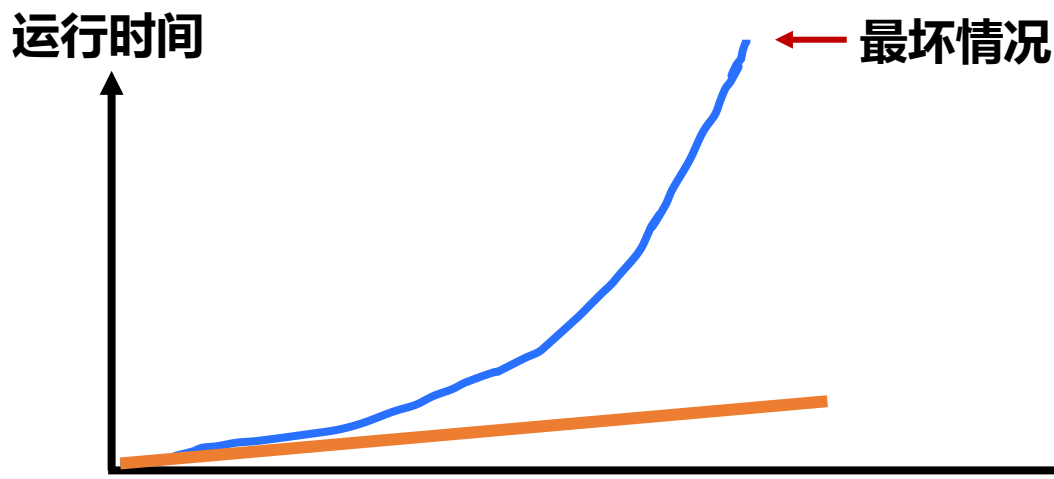
在经过大约 $52! = 80658\ 175\ 170\ 943\ 878\ 571\ 660\ 636\ 856\ 403\ 766\ 975\ 289\ 505\ 440\ 883\ 277\ 824\ 000\ 000000\ 000$ 次挑战之后，我们可能用**0分00秒**的排序时间成绩取代布拉德克。

2.3.2 操作计数（最好、最坏与平均情况）

参见课本：P040

□ 算法分析分最好、最坏和平均情况

输入情况	情况说明
最好情况	不常出现，不具有普遍性
最坏情况	确定上界，更具有一般性

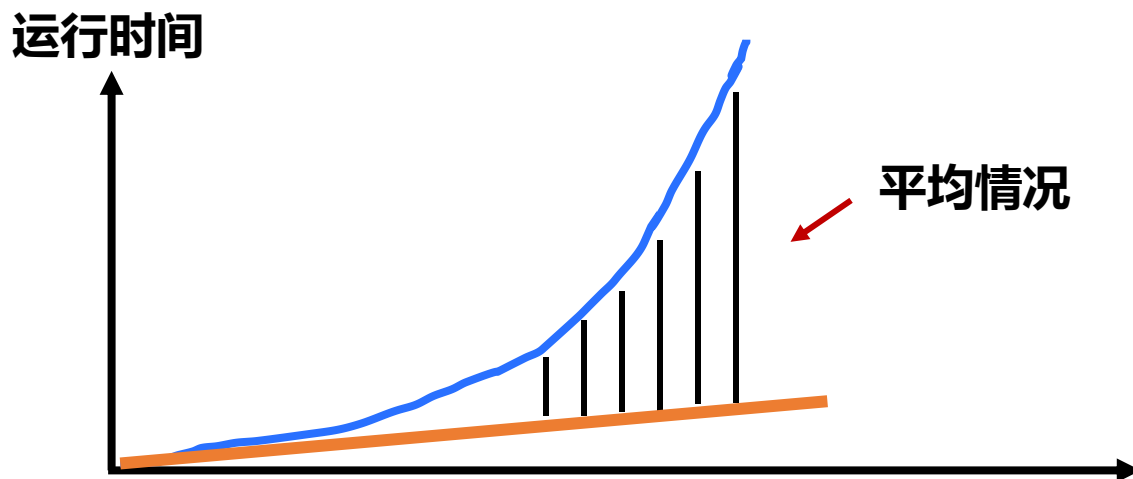


2.3.2 操作计数（最好、最坏与平均情况）

参见课本：P040

□ 算法分析分最好、最坏和平均情况

输入情况	情况说明
最好情况	不常出现，不具有普遍性
最坏情况	确定上界，更具有一般性
平均情况	情况复杂，分析难度较大

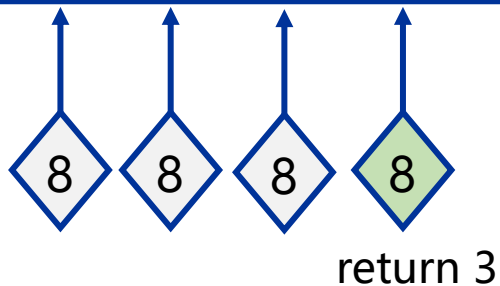


2.3.2 操作计数（最好、最坏与平均情况）

参见课本：P041

□ **例2-13 顺序查找：**从无序数组a [0,n-1]中搜索是否存在元素x，如果找到则返回x在数组中的位置，否则返回-1。

loc	0	1	2	3	4
num	4	6	2	8	1



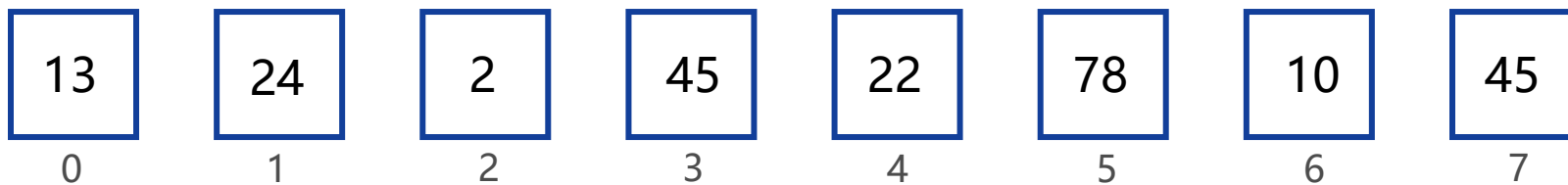
```
template <class T>
int SequentialSearch(T a[], const T& x,
int n)
{
    int i;
    for (i = 0; i < n && a[i] != x; i++);
    if (i == n) return -1;
    return i;
}
```


2.3.2 操作计数（最好、最坏与平均情况）

参见课本：P041

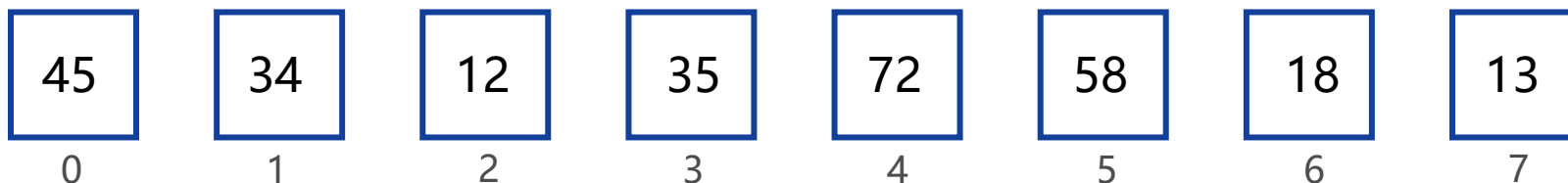
查找效率分析

最好情况



第一次比较就找到，复杂度是 $O(1)$

最差情况

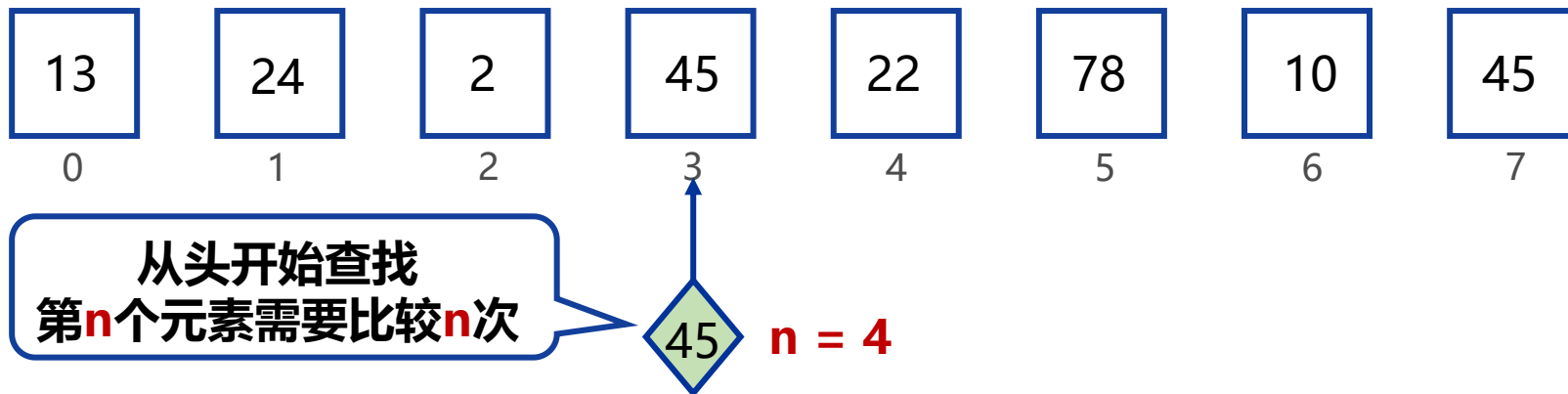


最后一次比较才找到，复杂度是 $O(n)$

2.3.2 操作计数（最好、最坏与平均情况）

参见课本：P041

查找效率分析



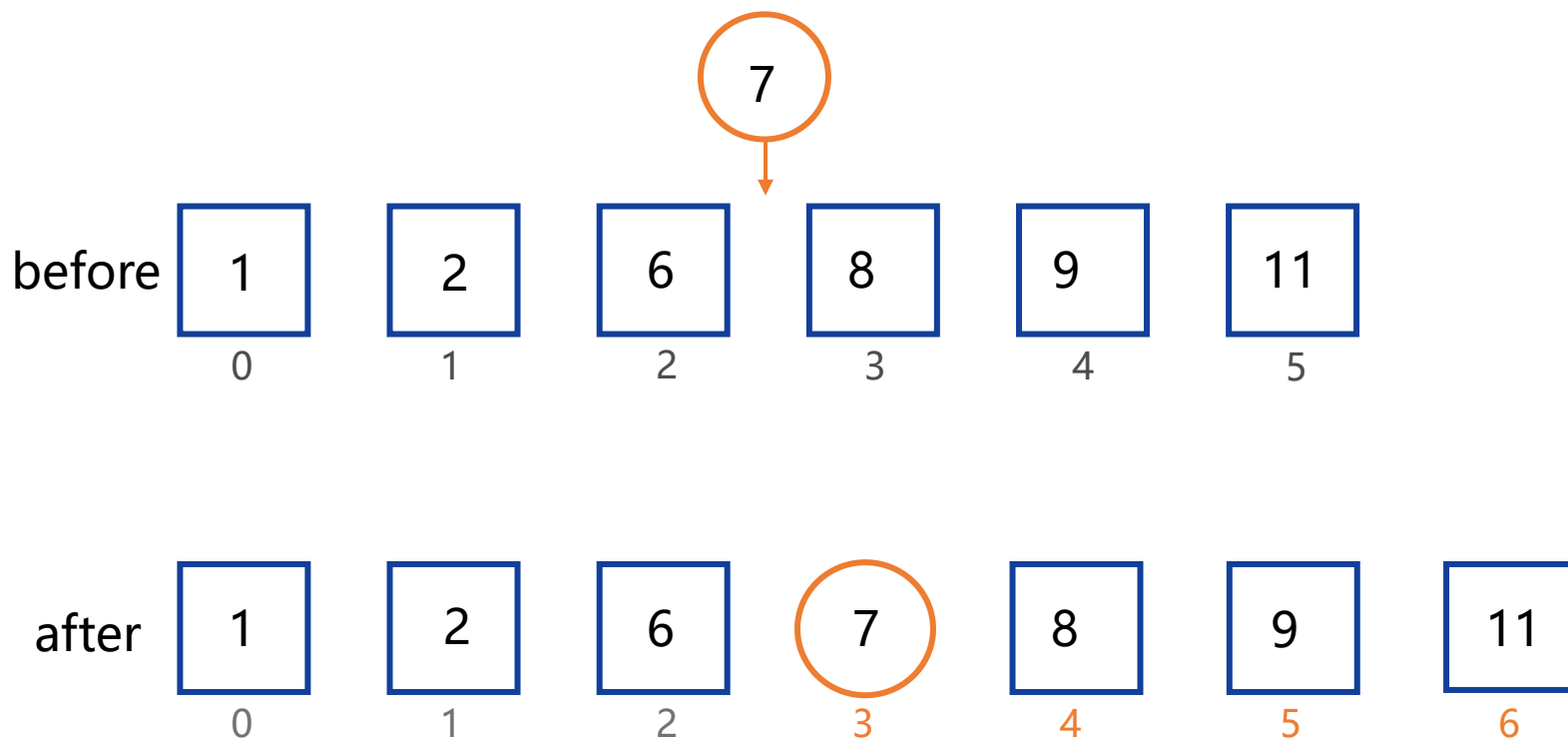
n个元素都被成功查找的比较次数总和为：
$$\sum_{i=1}^n i = \frac{n(n+1)}{2}$$

元素成功查找的**平均比较次数**：
$$\frac{1}{n} \sum_{i=1}^n i = \frac{n+1}{2}$$

2.3.2 操作计数（最好、最坏与平均情况）

参见课本：P041

□ **例2-14 向有序数组中插入元素：** 向一个有序数组 $a[0:n-1]$ 中插入一个元素。a中的元素在执行插入之前和插入之后都是按递增顺序排列。



2.3.2 操作计数（最好、最坏与平均情况）

参见课本：P041

```
template <class T>
void Insert (T a[], int & n, const T& x)
{ // Insert x into the sorted array a[0:n-1].
  // Assume a is of size > n.
```

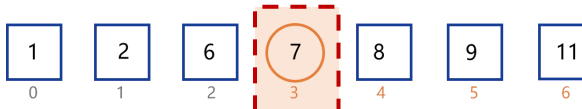
```
    int i;
```

```
    for (i = n-1; i >= 0 && x < a[i]; i--);
```

```
        a[i+1] = a[i]; ➔ after
```



```
        a[i+1] = x; ➡ after
```



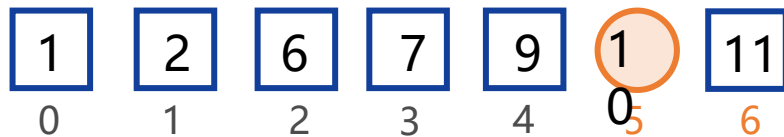
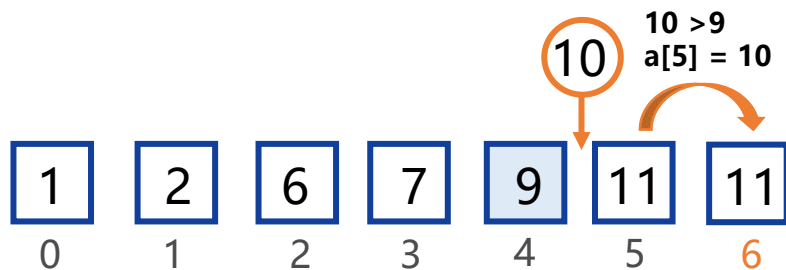
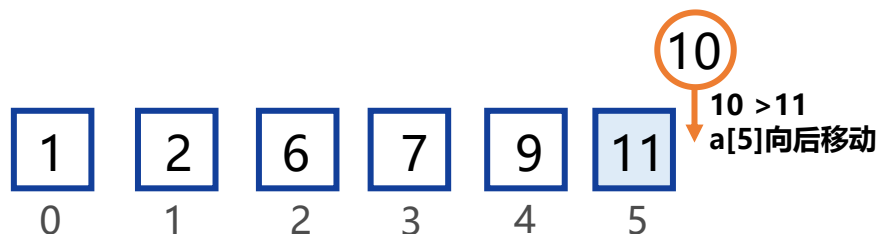
```
        n++; // one element added to a[]
```

```
    }
```

2.3.2 操作计数（最好、最坏与平均情况）

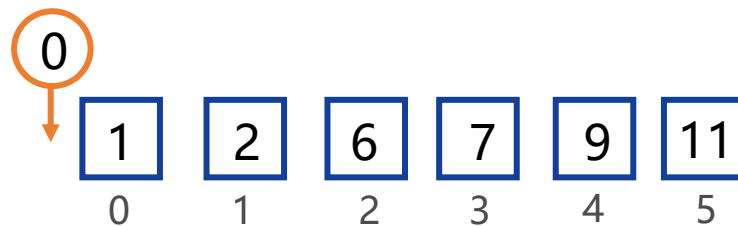
参见课本：P041

插入效率分析



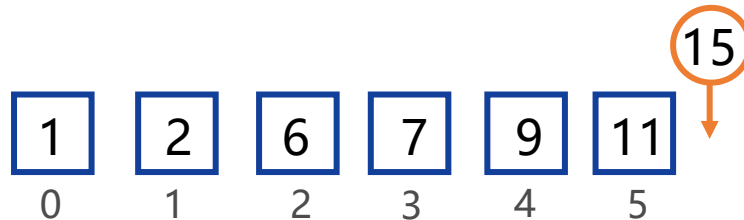
最差情况

x被插入到数组首部，最大的比较次数为n



最好情况

x被插入到数组尾部，最小的比较次数为1

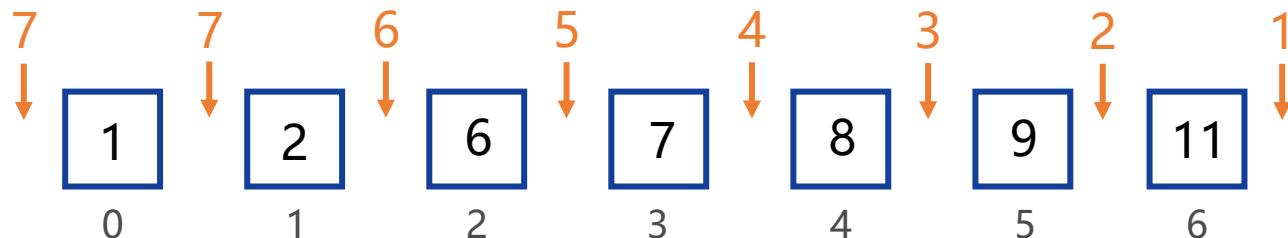


2.3.2 操作计数（最好、最坏与平均情况）

参见课本：P041

插入效率分析

假设x有相等的机会被插入到每一个位置上，共有8个位置



对于数组 $a[0:n-1]$,共有 $n + 1$ 个插入位置

$n + 1$ 个位置都被插入元素的比较次数总和为:

$$\sum_{i=1}^n i + n = \frac{n(n+1)}{2} + n$$

平均比较次数: $\frac{1}{n+1} (\sum_{i=1}^n i + n) = \frac{n}{2} + \frac{n}{n+1}$

x有相等的机会插入到每个位置这个假设合理么？

2.3.2 操作计数（最好、最坏与平均情况）

最坏情况：（常用）

- $T(n)$ = 算法在任何大小为 n 的输入上的**最大时间**。

平均情况：（有时）

- $T(n)$ = 算法在所有大小为 n 的输入上的**期望时间**。
- 需要假设输入的统计分布。

最好情况：（不可靠）

目录大纲

□ 算法分析概述

- 定义与分类
- 问题与问题实例
- 算法分析的原则

□ 空间复杂度

□ 时间复杂度



常用的分析方法

- 操作计数

- 程序步计数

- 最好、最坏与平均情况



渐进分析法



考察重点



熟练掌握技巧

2.3.3 执行步数

参见课本：P044

时间复杂度指程序执行时所用的时间

□ 在解析地分析时间复杂度时,使用以下两种时间单位并计算:

- 操作计数(operation count):找出一个或多个关键操作,确定这些关键操作所需要的执行时间
- (程序)步计数(step count):确定程序总的执行步数
- 要点:基本操作或程序步的执行时间必须是常数



2.3.3 执行步数

参见课本：P044

□ 计步法(step count): 统计程序的总“步数”，作为算法的时间复杂度

- 一个**程序步**是一个语法或语义上的程序片段，该片段由基本指令组成，执行时间视为常数

- 算术类指令：四则运算、取余、上下取整等

- 数据移动指令：装入、存储、复制等

- 控制指令：子程序调用与返回、条件转移等

- 划分程序步的方法不唯一，但数量级唯一

□ 算法的时间复杂度函数是单调递增的

2.3.3 执行步数

参见课本：P045

划分程序步的方法不唯一，但数量级唯一

程序步 (program step) 可以定义为一个语法或语义意义上的程序片段，该片段的执行时间**独立于实例特征 (n)**

return $a + b + b \times c + (a + b - c) / (a + b) + 4;$

可以被视为一个程序步，它的执行时间**独立于所选用的实例特征**

$x = y$

也被视为一个程序步，它的执行时间**独立于所选用的实例特征**

2.3.3 执行步数

□ **引例 顺序查找：** 从无序数组a [0,n-1]中搜索是否存在元素x，如果找到则返回x在数组中的位置，否则返回-1。

最好情况：

Statement	step	Frequency	steps
int SequentialSearch(T a[], T& x, int n)	0	0	0
{	0	0	0
int i;	1	1	1
for (int i = 0; i < n && a[i] != x; i++)	1	1	1
if (i == n) return -1;	1	1	1
return i;	1	1	1
}	0	0	0
Total			4

2.3.3 执行步数

□ **引例 顺序查找：**从无序数组a [0,n-1]中搜索是否存在元素x，如果找到则返回x在数组中的位置，否则返回-1。

最坏情况：

Statement	step	Frequency	steps
int SequentialSearch(T a[], T& x, int n)	0	0	0
{	0	0	0
int i;	1	1	1
for (int i = 0; i < n && a[i] != x; i++)	1	n+1	n+1
if (i == n) return -1;	1	1	1
return i;	1	0	0
}	0	0	0
Total			n+3

2.3.3 执行步数

参见课本：P045

如何计步？

创建一个全局变量count (初值为0),把 count 引入到程序语句之中

□ 例2-19 元素求和：计算a[0:n-1]中元素之和

```
template <class T>
T Sum (T a[], int n)
{ // 计算a[0:n-1]中元素之和
    T tsum = 0;
    count++; // 对应于tsum = 0
    for (int i = 0; i < n; i++) {
        count++; // 对应于for语句
        tsum += a[i];
        count++; // 对应于赋值语句
    }
    count++; // 对应于最后一个for语句
    count++; // 对应于return语句
    return tsum;
}
```

该程序仅用于说明
Step Count方法，
在应用该方法时并不
需要写这样的程序。

2.3.3 执行步数 (Step Table)

参见课本: P048

Step Table

s/e代表该语句执行后步数的变化(增量);

Frequency代表该语句的执行次数;

steps代表该语句在整个程序执行过程中引发的总步数。

Statement	s/e	Frequency	steps
T Sum(T a[], int n)	0	0	0
{	0	0	0
T tsum = 0;	1	1	1
for (int i = 0; i < n; i++)	1	n+1	n+1
tsum += a[i];	1	n	n
return tsum;	1	1	1
}	0	0	0
Total			2n+3

- 注意: for 语句的频率为 $n + 1$ 而不是 n , 循环变量 i 必须递增到 n , for 语句才能结束。

2.3.3 执行步数 (Step Table)

参见课本: P049

□ **程序2-23 前缀求和:** 用数组b[0:n-1]表示数组a[0:n-1]前n个元素的和

$$b[0] = a[0]$$

$$b[1] = a[0] + a[1]$$

$$b[2] = a[0] + a[1] + a[2]$$

$$b[3] = a[0] + a[1] + a[2] + a[3]$$

...

$$b[n-1] = a[0] + a[1] + \dots + a[n-2] + a[n-1]$$

2.3.3 执行步数 (Step Table)

参见课本：P049

□ 程序2-23 前缀求和：用数组b[0:n-1]表示数组a[0:n-1]前n个元素的和

```
void inef(T a[], T b[], int n)
{
    for (int j = 0; j < n; j++)
        b[j] = Sum(a, j+1);
}
```

已知Sum(a,m) 的执行步数为 $2m+3$ (上个例题)
赋值语句b[j] = sum(a, j+1)的执行步数为 (j+1代入m) :

$$2(j+1)+3+1 = 2j+6$$

j是变量

函数sum的值赋给b[j]需要一步

2.3.3 执行步数 (Step Table)

参见课本：P049

□ 程序2-23 前缀求和：用数组b[0:n-1]表示数组a[0:n-1]前n个元素的和

Statement	step	Frequency	steps
Void inef(T a[], T b[], int n)	0	0	0
{	0	0	0
for (int j = 0; j < n; j++)	1	n+1	n+1
b[j] = Sum(a, j+1);	2j+6	n	n(n+5)
}	0	0	0
Total			n ² +6n+1

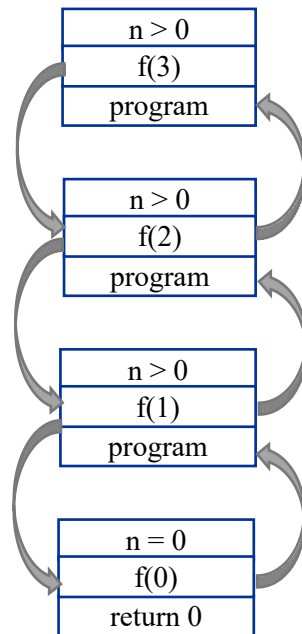
赋值语句的频率为n，该语句的总步数为 $\sum_{j=0}^{n-1} (2j + 6)$

2.3.3 执行步数（递归求和）

参见课本：P046

□ 例2-20 递归求和：计算数组 $a[0:n-1]$ 中元素之和

递去



不满足递归条件

归来

递归求和

Program

```
Template <class T>
T RSum(T a[], int n)
{ // Return sum of numbers a[0:n-1].
    if (n > 0)
        return Rsum(a, n-1) + a[n-1];
    return 0;
}
```

2.3.3 执行步数（递归求和）

参见课本：P046

□ 例2-20 递归求和：计算数组 $a[0:n-1]$ 中元素之和

当 $n=0$ 时



判断 $n > 0$ 是否成立，显然不满足递归条件

return 0;

递归求和 *Program*

```
Template <class T>
T RSum(T a[], int n)
{ // Return sum of numbers a[0:n-1].
    if (n > 0)
        return Rsum(a, n-1) + a[n-1];
    return 0;
}
```

2.3.3 执行步数（递归求和）

参见课本：P046

□ 例2-20 递归求和：计算数组a[0:n - 1]中元素之和

当 $n > 0$ 时



判断 $n > 0$ 是否成立

$$T(0)=2,$$

$$T(1) = T(0)+2,$$

$$T(2) = T(1)+2,$$

...

$$T(n) = T(n-1)+2,$$

$$\begin{aligned} T(n) &= T(n-1)+2 \\ &= T(n-2) + 4 = \dots = 2(n+1) \end{aligned}$$

递归求和 *Program*

```
Template <class T>
T RSum(T a[], int n)
{ // Return sum of numbers a[0:n-1].
    if (n > 0)
        return Rsum(a, n-1) + a[n-1];
    return 0;
}
```

在分析递归程序时，使用解递归方程的方法

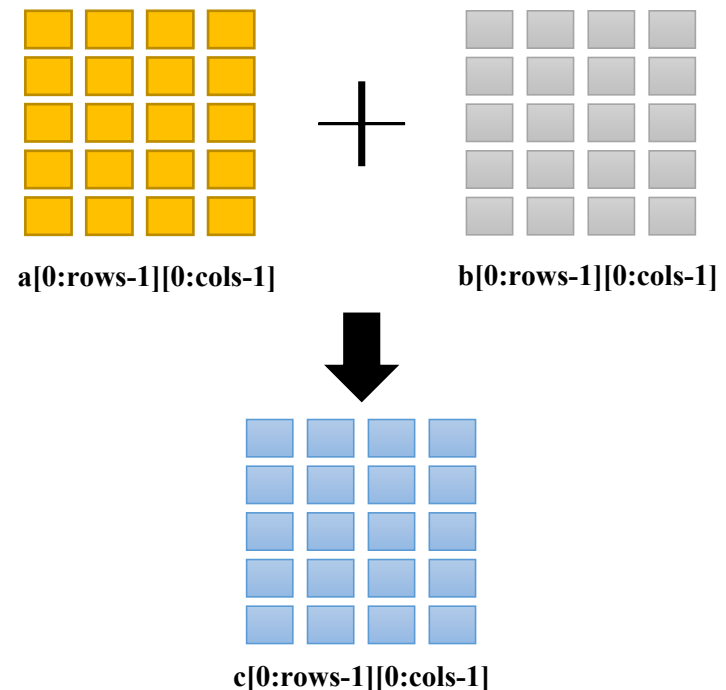
2.3.3 执行步数（矩阵相加）

参见课本：P046

□ **例2-21 矩阵相加：**考察程序2-19, 它把存储在二维数组a[0:rows-1][0:cols-1]和 b[0:rows-1][0:cols-1]中的两个矩阵相加。

● 程序 2-19 如下：

```
Template <class T>
void Add(T * * a, T * * b, T * * c,
int rows, int cols)
{ // 矩阵a和b相加得到矩阵c
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            c[i][j] = a[i][j] + b[i][j];
}
```



2.3.3 执行步数（矩阵相加）

参见课本：P046

□ **例2-21 矩阵相加：**考察程序2-19, 它把存储在二维数组a[0:rows-1][0:cols-1]和 b[0:rows-1][0:cols-1]中的两个矩阵相加。

Statement	step	Frequency	steps
void Add(T * * a, T * * b, T * * c,	0	0	0
int rows, int cols)	0	0	0
{// 矩阵a和b相加得到矩阵c	1	rows+1	rows+1
for (int i = 0; i < rows; i++)	1	rows*(cols+1)	rows*cols+rows
for (int j = 0; j < cols; j++)	1	rows*cols	rows*cols
c[i][j] = a[i][j] + b[i][j];	0	0	0
}			
Total		2rows · cols+2rows+1	

目录大纲

□ 算法分析概述

- 定义与分类
- 问题与问题实例
- 算法分析的原则

□ 空间复杂度

□ 时间复杂度



常用的分析方法

- 操作计数
- 程序步计数

- 最好、最坏与平均情况



渐进分析法



考察重点



熟练掌握技巧

2.4 渐进符号（渐进分析的必要性）

参见课本：P055

Q：操作计数（operation count）和程序步计数（step count）的方法精准么？

- 操作计数：把注意力集中在某些“关键”的操作上，而忽略了所有其他操作。
- 程序步计数：指令 $x = y$ 和 $x = y + z + (x / y)$ 都可以被称为一步。

2.4 渐进符号（渐进分析的必要性）

参见课本：P055

- 有两个程序的时间复杂性分别为 $c_1n^2 + c_2n$ 和 c_3n
 - $n \rightarrow \infty$, 复杂性为 c_3n 的程序将比复杂性为 $c_1n^2 + c_2n$ 的程序运行更快
 - 对于比较小的 n 值, 两者都有可能成为较快的程序（取决于 c_1 、 c_2 和 c_3 ）

讨论算法和数据结构时, 关注算法复杂度是怎样随着问题规模增大而增加, 尤其当问题具有很大输入时

2.4 渐进符号

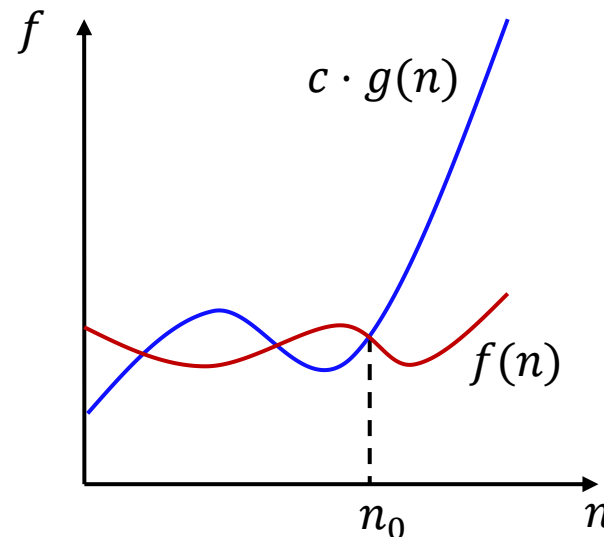
参见课本：P055

- 通过忽略常数因子和低阶项，关注算法复杂度的增长量级。这种分析方法叫**渐近分析**
 - 渐近符号 (O , Θ , Ω) 是算法分析中使用最多的符号
 - Θ ：渐近等于，渐近紧界；
 - O ：渐近大于，渐近上界；
 - Ω ：渐近小于，渐近下界。
 - 渐近分析中忽略常数因子和低阶项，只关注剩下的内容
 - 例如，如果 $f(n) = 6n\log_2 n + 6n$ ，则 $f(n) = \Theta(n\log n)$
- 注： $\oplus$ 可以表示 O 、 Θ 、 Ω 中的任何一个。

2.4.1 渐进上界

参见课本：P056

□ **定义 [大写O符号]** $f(n) = O(g(n))$ 当且仅当存在正的常数 c 和 n_0 , 使得对于所有的 $n \geq n_0$, 有 $f(n) \leq cg(n)$ 。



对于足够大的 n , 函数 $g(n)$ 是函数 $f(n)$ 渐近意义上的上界

2.4.1 渐进上界

参见课本：P056-P057

□ **定义 [大写O符号]** $f(n) = O(g(n))$ 当且仅当存在正的常数 c 和 n_0 , 使得对于所有的 $n \geq n_0$, 有 $f(n) \leq cg(n)$ 。

• **例2-24 线性函数:** 考察 $f(n) = 3n + 2$

➤ 当 $n \geq 2$ 时, $3n + 2 \leq 3n + n = 4n$, $f(n) = O(n)$

• **例2-25 平方函数:** 考察 $f(n) = 10n^2 + 4n + 2$

➤ 当 $n \geq 2$ 时, 有 $f(n) \leq 10n^2 + 5n$;

➤ 当 $n \geq 5$ 时, 有 $5n \leq n^2$, $f(n) = 10n^2 + 5n \leq 11n^2$, $f(n) = O(n^2)$

2.4.1 渐进上界

参见课本：P057

□ **定义 [大写O符号]** $f(n) = O(g(n))$ 当且仅当存在正的常数 c 和 n_0 , 使得对于所有的 $n \geq n_0$, 有 $f(n) \leq cg(n)$ 。

- **例2-26 指数函数:** 考察 $f(n) = 6 \times 2^n + n^2$
 - 当 $n \geq 4$ 时, $n^2 \leq 2^n$, $f(n) \leq 6 \times 2^n + 2^n = 7 \times 2^n$, $f(n) = O(2^n)$
- **例2-27 常数函数:** 考察 $f(n) = 2033$
 - 当 $n \geq 0$ 时, $f(n) \leq 2033 \times 1$, $f(n) = O(1)$

2.4.1 渐进上界

参见课本：P057

□ **定义 [大写O符号]** $f(n) = O(g(n))$ 当且仅当存在正的常数 c 和 n_0 , 使得对于所有的 $n \geq n_0$, 有 $f(n) \leq cg(n)$ 。

• **例2-28 松散界限:** 考察 $f(n) = 3n + 2$

- 存在当 $n \geq 2$ 时, $3n + 2 \leq 3n^2$, $f(n) = O(n^2)$, 但不是**最小**上限;
- 可以找到一个更小的函数 ($f(n) = O(n)$) 来满足大O定义;
- 用次数低的项目替换次数高的项目, 直到剩下**一个单项**为止。

• **例2-29 错误界限:** 证明 $f(n) = 3n + 2 \neq O(1)$

- 假定存在 c 及 n_0 , 使得对于所有的 $n \geq n_0$, 有 $3n + 2 < c$, $n < (c - 2)/3$, 当 $n > \max\{n_0, (c - 2)/3\}$ 时, 将出现矛盾。

2.4.1 渐进上界

参见课本：P058

□ **定理2-2 [大O比率定理]** 对于函数 $f(n)$ 和 $g(n)$ ，若存在

$\lim_{n \rightarrow \infty} f(n)/g(n)$ ，则 $f(n) = O(g(n))$ 当且仅存在确定的

常数 c ($0 \leq c < \infty$) 成立，有 $\lim_{n \rightarrow \infty} \left| \frac{f(n)}{g(n)} \right| = c$ 。

• **例2-31:**

- 考察 $f(n) = 3n + 2$ ， $\lim_{n \rightarrow \infty} \left| \frac{3n+2}{n} \right| = 3$ ， $f(n) = O(n)$ ；
- 考察 $f(n) = 10n^2 + 4n + 2$ ， $\lim_{n \rightarrow \infty} \left| \frac{10n^2+4n+2}{n^2} \right| = 10$ ， $f(n) = O(n^2)$ ；
- 考察 $f(n) = 6 * 2^n + n^2$ ， $\lim_{n \rightarrow \infty} \left| \frac{6*2^n+n^2}{2^n} \right| = 6$ ， $f(n) = O(2^n)$ 。

2.4.1 渐进上界

参见课本：P058

□ **定理2-2 [大O比率定理]** 对于函数 $f(n)$ 和 $g(n)$ ，若存在

$\lim_{n \rightarrow \infty} f(n)/g(n)$ ，则 $f(n) = O(g(n))$ 当且仅存在确定的

常数 c ($0 \leq c < \infty$) 成立，有 $\lim_{n \rightarrow \infty} \left| \frac{f(n)}{g(n)} \right| = c$ 。

• **例2-31:**

➤ 考察 $f(n) = 2n^2 - 3$ ， $\lim_{n \rightarrow \infty} \left| \frac{2n^2 - 3}{n^4} \right| = 0$ ， $f(n) = O(n^4)$ ；

➤ 考察 $f(n) = 3n^2 + 5$ ， $\lim_{n \rightarrow \infty} \left| \frac{3n^2 + 5}{n} \right| = \infty$ ， $f(n) \neq O(n)$ 。

2.4.1 渐进上界（常用的渐进函数）

□ 对 $f(n) = O(g(n))$ ，其中 $g(n)$ 称为**渐进函数**，通常使用比较简单的函数形式。比较典型的形式是含有 n 的单个项。

函数	名称
1	常数
$\log n$	对数
n	线性
$n \log n$	n 个 $\log n$
n^2	平方
n^3	立方
2^n	指数
$n!$	阶乘

常用渐进函数

2.4.1 渐进上界（渐进分析）

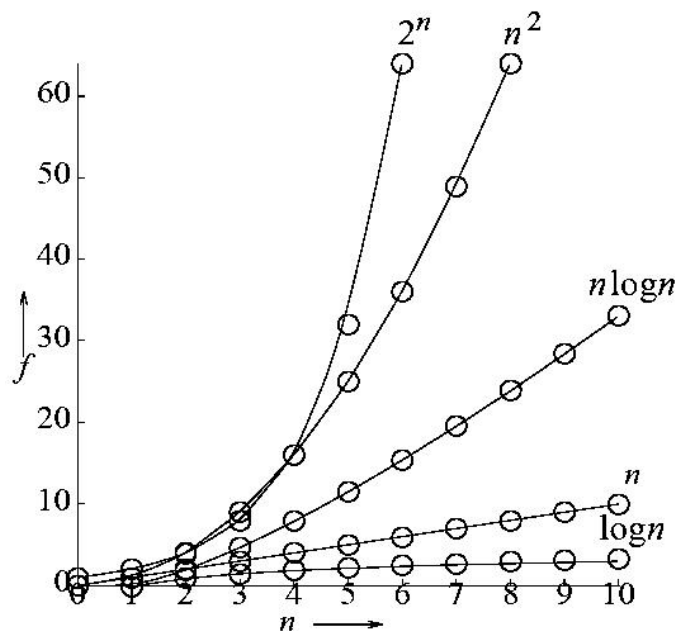
参见课本：P067

□ 算法可以在巨大输入下工作吗？

$\log n$	n	$n \log n$	n^2	n^3	2^n
0	1	0	1	1	2
1	2	2	4	8	4
2	4	8	16	64	16
3	8	24	64	512	256
4	16	64	256	4096	65536
5	32	160	1024	32768	4294967296

各函数值表

□ 随 n 的增加 $T(n)$ 的增长率变化



各函数曲线图

2.4.1 渐进上界（不同输入的运行时间）

参见课本：P068

	n	$n \log_2 n$	n^2	n^3	n^{10}	2^n	$n!$
$n=10$	$.01\mu\text{s}$	$.03\mu\text{s}$	$.1\mu\text{s}$	$1\mu\text{s}$	10s	$1\mu\text{s}$	4 ms
$n=30$	$.03\mu\text{s}$	$.15\mu\text{s}$	$.9\mu\text{s}$	$27\mu\text{s}$	6.83d	1s	10^{22} y
$n=50$	$.05\mu\text{s}$	$.28\mu\text{s}$	$2.5\mu\text{s}$	$125\mu\text{s}$	3.1y	13d	
$n=100$	$.10\mu\text{s}$	$.66\mu\text{s}$	$10\mu\text{s}$	1ms	3171y	$4 * 10^{13}\text{y}$	
$n=10^3$	$1\mu\text{s}$	$9.96\mu\text{s}$	1ms	1s	$3.17 * 10^{13}\text{y}$	$32 * 10^{283}\text{y}$	
$n=10^4$	$10\mu\text{s}$	$130\mu\text{s}$	100ms	16.67m	$3.17 * 10^{23}\text{y}$		
$n=10^5$	$100\mu\text{s}$	1.66ms	10s	11.57d	$3.17 * 10^{33}\text{y}$		
$n=10^6$	1ms	19.92ms	16.67m	31.71y	$3.17 * 10^{43}\text{y}$		

- 上表是程序在一台 10^9 条指令/秒的计算机上的运行时间。第一行是对应的时间复杂度函数。
- 从表中结果看，对于 $n > 100$ 的输入规模，只有那些复杂度比较小的程序（如 $O(n^3)$ ）才是可行的。实际情况也是如此。

2.4.2 渐进下界

参见课本：P058

□ **定义** [Ω 符号] $f(n) = \Omega(g(n))$ 当且仅当存在正的常数 c 和 n_0 , 使得对于所有的 $n \geq n_0$, 有 $f(n) \geq cg(n)$ 。

□ **定义** [大写O符号] $f(n) = O(g(n))$ 当且仅当存在正的常数 c 和 n_0 , 使得对于所有的 $n \geq n_0$, 有 $f(n) \leq cg(n)$ 。

2.4.2 渐进下界

参见课本：P058

□ 例2-32：考察 $f(n) = 3n + 2$

- 对于所有的 n ，有 $f(n) = 3n + 2 > 3n$ ， $f(n) = \Omega(n)$ ；
- 当 $n \geq 2$ 时， $3n < 3n + 2 \leq 4n$

左侧 ' $<$ ' 确定渐进下界，右边 ' $<$ ' 确定渐进上界；

- $f(n) = \Omega(n)$ ， $f(n) = O(n)$ 。

Q: 上面 $f(n) = \Omega(1)$ 对不对？

2.4.3 渐进紧界

参见课本：P059

Q: 上面 $f(n) = \Omega(1)$ 对不对?

- 正确，是松散界限；
- 如果 $f(n) = O(g(n))$ 上同时有 $f(n) = \Omega(g(n))$ ，则记作 $f(n) = \Theta(g(n))$ ，称 $f(n)$ 与 $g(n)$ 同阶。

□ **定义** [Θ 符号] $f(n) = \Theta(g(n))$ 当且仅当存在正的常数 c_1 、 c_2 和 n_0 ，使得对于所有的 $n \geq n_0$ ，有 $c_1 g(n) \leq f(n) \leq c_2 g(n)$ 。

2.4 渐进符号 (O 、 Ω 、 Θ)

□ 对于函数 $f(n)$ 和 $g(n)$, 若存在 $\lim_{n \rightarrow \infty} f(n)/g(n)$

□ 对 $\lim_{n \rightarrow \infty} \left| \frac{f(n)}{g(n)} \right| = c$:

• 若 $0 \leq c < \infty$, 则 $f(n) = O(g(n))$;

渐进上界

• 若 $0 < c \leq \infty$, 则 $f(n) = \Omega(g(n))$;

渐进下界

• 若 $0 < c < \infty$, 则 $f(n) = \Theta(g(n))$;

渐进紧界

2.4 渐进符号 (Big-O 更常用)

- 由于算法设计师非常注重运行时间的保证（重视上界），因此更倾向使用 **big-O 表示法**，即使big- Θ 表示法更精确。
- 使用 “Big-O” 记法描述例 3.1 的结果：
 - $10n + 7 = O(3n^2 + 2n + 6)$;
 - $100n^3 - 3 = O(8n^4 + 9n^2)$;
 - $12n + 6 = O(6n + 2)$;
 - $3n^2 + 2n + 6 \neq O(10n + 7)$;
 - $8n^4 + 9n^2 \neq O(100n^3 - 3)$ 。

2.4 渐进符号 (Big-O 更常用)

- 由于算法设计师非常注重运行时间的保证（重视上界），因此更倾向使用 **big-O 表示法**，即使big- Θ 表示法更精确。
- 使用 “Big-O” 记法描述例 3.2 的结果：
 - $10n + 7 = O(n)$;
 - $100n^3 - 3 = O(n^3)$;
 - $12n + 6 = O(n)$;
 - $3n^2 + 2n + 6 \neq O(n)$;
 - $8n^4 + 9n^2 \neq O(n^3)$ 。

2.4 渐进符号 (Big-O 更常用)

□ 使用 “Big-O” 记法描述下列例题的结果：

例题	$T(n)$	big-O 表示法
例2.15	$T_{Rearrange}(n) = 2(n - 1)$	$O(n)$
例2.16	$T_{SelectionSort}(n) = n(n - 1)/2$	$O(n^2)$
例2.17	$T_{Bubble}(n) = n(n - 1)/2$	$O(n^2)$
例2.18	$T_{InsertionSort}(n) = n + 3$	$O(n)$
例2.19	$T_{Sum}(n) = 2n + 3$	$O(n)$
例2.20	$T_{Inef}(n) = n^2 + 6n + 1$	$O(n^2)$
例2.21	$T_{Rsum}(n) = 2(n + 1)$	$O(n)$
例2.22	$T_{Add}(r, s) = 2rs + 2r + 1$	$O(rs)$

2.4 渐进符号 (Big-O 更常用)

□ 考察 $f(m, n) = 3m^2n + m^3 + 10mn + 2n^2$

- $10mn \leq 3m^2n$, 在其余项中没有有一个渐近小于另一个。
- 删去 $10mn$, 剩余项的系数改为1, 得到

$$f(m, n) = O(m^2n + m^3 + 2n^2)$$

- “Big-O” 不是紧界, 所以上式也可以表示成

$$f(m, n) = O(m^3n^2)$$

课堂习题（两分钟）

□ 判断对错：

- $10n + 7 = \Omega(n)$
- $100n^3 - 3 = \Omega(n^3)$
- $12n + 6 = \Omega(n)$
- $3n^3 + 2n + 6 = \Omega(n)$
- $8n^4 + 9n^2 = \Omega(n^3)$
- $3n^3 + 2n + 6 = \Omega(n^5)$
- $8n^4 + 9n^2 = \Omega(n^5)$
- $10n + 7 = \Theta(n)$
- $100n^3 - 3 = \Theta(n^3)$
- $12n + 6 = \Theta(n)$
- $3n^3 + 2n + 6 = \Theta(n)$
- $8n^4 + 9n^2 = \Theta(n^3)$
- $3n^3 + 2n + 6 = \Theta(n^5)$
- $8n^4 + 9n^2 = \Theta(n^5)$

课堂习题（两分钟）

□ 判断对错：

✓ • $10n + 7 = \Omega(n)$

✓ • $100n^3 - 3 = \Omega(n^3)$

✓ • $12n + 6 = \Omega(n)$

✓ • $3n^3 + 2n + 6 = \Omega(n)$

✓ • $8n^4 + 9n^2 = \Omega(n^3)$

✗ • $3n^3 + 2n + 6 = \Omega(n^5)$

✗ • $8n^4 + 9n^2 = \Omega(n^5)$

✓ • $10n + 7 = \Theta(n)$

✓ • $100n^3 - 3 = \Theta(n^3)$

✓ • $12n + 6 = \Theta(n)$

✗ • $3n^3 + 2n + 6 = \Theta(n)$

✗ • $8n^4 + 9n^2 = \Theta(n^3)$

✗ • $3n^3 + 2n + 6 = \Theta(n^5)$

✗ • $8n^4 + 9n^2 = \Theta(n^5)$

2.4.5 渐进特性（求和公式）

参见课本：P061

求和公式的渐近表示

$f(n)$	渐进表示
c	$\Theta(1)$
$\sum_{i=1}^n 1/i$	$\Theta(\log n)$
$\sum_{i=0}^k c_i n^i$	$\Theta(n^k)$
$\sum_{i=0}^n i$	$\Theta(n^2)$
$\sum_{i=0}^n i^2$	$\Theta(n^3)$
$\sum_{i=0}^n i^k, k > 0$	$\Theta(n^{k+1})$
$\sum_{i=0}^n r^i, r > 1$	$\Theta(r^n)$
$n!$	$\Theta(\sqrt{n} \cdot (n/e)^n)$

注：• 斯特林公式 $\lim_{n \rightarrow +\infty} \frac{n!}{\sqrt{2\pi n} \left(\frac{n}{e}\right)^n} = 1$

- [斯特林公式 百度百科 \(baidu.com\)](https://baike.baidu.com/item/%E6%96%87%E4%BF%9C%E5%85%B6%E5%BC%8F/1077111)

2.4.5 渐进特性（推理规则）

参见课本：P061

□ 推理规则

- **I1** $\{f(n) = \oplus (g(n))\} \Rightarrow \sum_{n=a}^b f(n) = \oplus (\sum_{n=a}^b g(n))$

- **I2** $\{f_i(n) \oplus (g_i(n)), 1 \leq i \leq k\} \Rightarrow$

$$\sum_{i=1}^k f_i(n) = \oplus (\max_{1 \leq i \leq k} \{g_i(n)\})$$

- **I3** $\{f_i(n) \oplus (g_i(n)), 1 \leq i \leq k\} \Rightarrow$

$$\prod_{i=1}^k f_i(n) = \oplus (\prod_{i=1}^k g_i(n))$$

- **I1**: 带n循环里的多级调用, 例如: for 循环。
- **I2**: 多步调用, 例如: 一个程序里的不同步骤。
- **I3**: 循环里的多级调用, 递归, 例如: 多级for循环。

2.4.5 渐进特性（推理规则）

参见课本：P062

□ 回顾【元素求和】：计算 $a[0:n-1]$ 中元素之和

Statement	step	Frequency	steps
T Sum(T a[], int n)	0	0	0
{	0	0	0
T tsum = 0;	1	1	1
for (int i = 0; i < n; i++)	1	n+1	n+1
tsum += a[i];	1	n	n
return tsum;	1	1	1
}	0	0	0
Total			2n+3

- ‘tsum += a[i];’ 该语句从 $i=0$ 到 $i=n-1$ 执行了 n 次， $i=n$ 时跳出循环。

2.4.5 渐进特性（推理规则）

□ 回顾【前缀求和】：求数组前n个元素的和

Statement	step	Frequency	steps
Void inef(T a[], T b[], int n)	0	0	0
{	0	0	0
for (int j = 0; j < n; j++)	1	n+1	n+1
b[j] = Sum(a, j+1);	2j+6	n	n(n+5)
}	0	0	0
Total			n^2+6n+1

赋值语句的频率为n，该语句的总步数为 $\sum_{j=0}^{n-1} (2j + 6)$

使用 for 循环



基于 I1 规则

2.4.5 渐进特性（推理规则）

□ 回顾【前缀求和】：求数组前n个元素的和

Statement	step	Frequency	steps	steps
Void inef(T a[], T b[], int n)	0	0	0	$\Theta(0)$
{	0	0	0	$\Theta(0)$
for (int j = 0; j < n; j++)	1	n+1	n+1	$\Theta(n)$
b[j] = Sum(a, j+1);	2j+6	n	n(n+5)	$\Theta(n^2)$
}	0	0	0	$\Theta(0)$
Total			n^2+6n+1	$\Theta(n^2)$

多步调用的时候
选择最大



基于 I2 规则

2.4.5 渐进特性（推理规则）

参见课本：P062

□ 回顾【矩阵相加】：矩阵a和b相加得到矩阵c

Statement	step	Frequency	steps
void Add(T * * a, T * * b, T * * c,	0	0	0
int rows, int cols)	0	0	0
{// 矩阵a和b相加得到矩阵c	0	0	0
for (int i = 0; i < rows; i++)	1	rows+1	rows+1
for (int j = 0; j < cols; j++)	1	rows*(cols+1)	rows*cols+rows
c[i][j] = a[i][j] + b[i][j];	1	rows*cols	rows*cols
}	0	0	0
Total		2rows · cols+2rows+1	

使用两级 for 循环



基于 I3 规则

2.4.5 渐进特性（推理规则）

参见课本：P062

□ 回顾【递归求和】：计算数组a[0:n - 1]中元素之和

Statement	step	Frequency	steps
T RSum(T a[], int n)	0	0	$\Theta(0)$
{// Return sum of numbers a[0:n-1].	0	0	$\Theta(0)$
if (n > 0)	1	n+1	$\Theta(n)$
return Rsum(a, n-1) + a[n-1];	1	n	$\Theta(n)$
return 0;	1	1	$\Theta(1)$
}	0	0	$\Theta(0)$
Total			$\Theta(n)$

2.4.5 渐进特性（推理规则）

参见课本：P063

□ 回顾【顺序查找】：从无序数组a [0,n-1]中搜索是否存在元素x，如果找到则返回x在数组中的位置，否则返回-1。

Statement	step	Frequency	steps
int SequentialSearch(T a[], T& x, int n)	0	0	$\Theta(0)$
{	0	0	$\Theta(0)$
int i;	1	1	$\Theta(1)$
for (int i = 0; i < n && a[i] != x; i++)	1	$\Omega(1)O(n)$	$\Omega(1)O(n)$
if (i == n) return -1;	1	1	$\Theta(1)$
return i;	1	$\Omega(0)O(1)$	$\Omega(0)O(1)$
}	0	0	$\Theta(1)$
Total			$\Theta(n)$

$$t_{\text{SequentialSearch}}(n) = O(n) \quad t_{\text{SequentialSearch}}(n) = \Omega(1)$$

算法的分类

精确算法

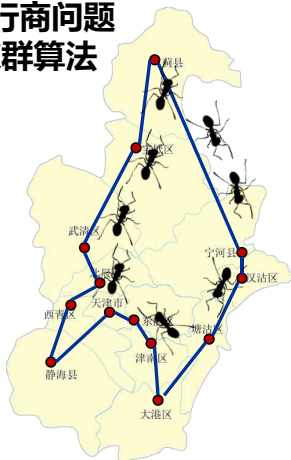
冒泡排序

1	6	2	3	4	5
1	2	6	3	4	5
1	2	3	6	4	5
1	2	3	4	6	5
1	2	3	4	5	6

总能保证求得问题**最优解**的算法

启发式算法

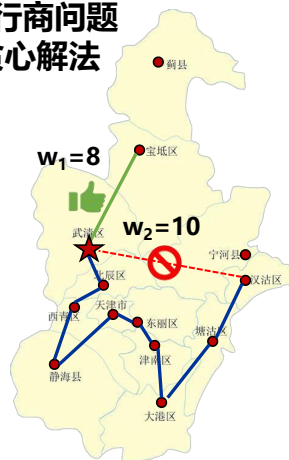
旅行商问题
-蚁群算法



通过某种规则、
简化策略或智能
猜测来**减少问题
求解时间**

近似算法

旅行商问题
-贪心解法



寻找最优化问题的
近似解而不是
最优解

随机算法

随机产生暴击



在执行过程中需
做出某些**随机选
择**

多项式时间算法

- 如果算法的最坏情形时间复杂度 $t(n) = O(n^k)$, 则称该算法为多项式复杂度的算法或有多项式界的算法
- 如果算法的最坏情形时间复杂度 $t(n)$ 不能用多项式限界, 则称该算法为指数复杂度的算法。这类算法可认为计算上不可行的算法

NP-hard 问题

- 如果一个问题有多项式界的算法称该问题属于多项式类 P 问题
- 很多实际上有意义的问题找不到有多项式界的算法。则称这些问题是 NP-hard 问题，即问题本身难。
- 上述难问题只能通过近似算法或启发式算法求解。

本章小结

□ (PPT04-12) 算法分析概述

- 测量与**解析**的方法
- 问题与问题实例 (**实例特征 n**)
- 算法分析的原则

分析算法的运行时间应独立于机器

□ (PPT14-22) 空间复杂度 $S(n)$

- 指令、数据、环境栈空间
- $S(n) = c + S_p(n)$

忽略与实例特征无关的空间需求量



考察重点



熟练掌握技巧

□ (PPT24-73) 时间复杂度 $T(n)$



方法一：操作计数



方法二：程序步技术 (Step Table)

- 三种情况分析对比

最好、**最差**与平均情况

□ (PPT75-104) 渐进分析法



分类与判定方法

渐进上界、渐进下界与渐进紧界



渐进函数的推理规则

多级调用、多步调用

- NP-hard与多项式问题

本章作业

□ 第二程序的性能

- Q15、Q16、Q18、Q20、Q24;

(课本P52-P55)

- Q37(2)(4)(6)(7)(8)(9)(13)

(课本P66)

算法分析概论

1. 设 n 是描述问题规模的非负整数, 下面程序片段的时间复杂度是(A).

$x=2;$

$\text{while } (x < n/2)$

$x=2*x;$

A. $O(\log n)$ B. $O(n)$ C. $O(n \log n)$ D. $O(n^2)$

算法分析概论

2. 求整数 $n(n \geq 0)$ 阶乘的算法如下, 其时间复杂度是(B).

```
int fact(int n)
{
    if (n <= 1) return 1;
    return n * fact(n-1);
}
```

A. $O(\log n)$ B. $O(n)$ C. $O(n \log n)$ D. $O(n^2)$

算法分析概论

3. 下列程序段的时间复杂度是(C).

```
count=0;
```

```
for (k=1; k<=n; k*=2)
```

```
    for (j=1; j<=n; j++) count++;
```

A. $O(\log n)$ B. $O(n)$ C. $O(n \log n)$ D. $O(n^2)$

算法分析概论

4. 算法的计算量的大小称为计算的(B).
A. 效率 B. 复杂性 c. 现实性 D. 难度
5. 计算机算法指的是(C), 它必须具备(B)这三个特性。
- (1) A. 计算方法 B. 排序方法
C. 解决问题的步骤 D. 调度方法
- (2) A. 可执行性、可移植性、可扩充性
B. 可执行性、确定性、有穷性
C. 确定性、有穷性、稳定性
D. 易读性、稳定性、安全性

算法分析概论

6. 算法的时间复杂度取决于(C).

A. 问题的规模 B. 待处理数据的初态 C. A和B

7. 设计一个 “好” 的算法应该达到的目标是
(B、D).

A. 可行的 B. 健壮的 C. 无二义性 D. 可读性好

算法分析概论

8. 计算算法的时间复杂度是属于一种(B)。
- A. 事前统计的方法 B. 事前分析估算的方法
C. 事后统计的方法 D. 事后分析估算的方法
9. 算法分析的目的是(C)。
- A. 找出数据结构的合理性
B. 研究算法中的输入和输出的关系
C. 分析算法的效率以求改进
D. 分析算法的易懂性和文档性

算法分析概论

10. 下面程序的算法复杂性为(B)。

```
for (int i=0; i<m; i++)  
    for (int j=0; j<n; j++)  
        a[i][j]=i*j;
```

A. $O(n^2)$ B. $O(m * n)$ C. $O(m^2)$ D. $O(m + n)$

算法分析概论

11. 在下列算法中, “ $x=x*2$ ” 的执行次数是(A)。

```
int suanfa1 (int n)
{
    int i, j, x=1;
    for (i=0; i<n; i++)
        for (j=i; j<n; j++) x=x*2;
    return x;
}
```

A. $n(n+1)/2$

B. $n\log_2 n$

C. n^2

D. $n(n-1)/2$

算法分析概论

12. 执行下列算法suanfa2(1000), 输出结果是(**C**)。

```
void suanfa2 (int n)
{
    int i=1;
    while (i<=n) i*=2;
    printf("%d", i);
}
```

A. 2000

B. 512

C. 1024

D. 2^{1000}

算法分析概论

13. 当 n 足够大时下述函数中渐进时间最小的是 (B)。

A. $T(n) = n\log_2 n - 1000\log_2 n$

B. $T(n) = n\log_2 3 - 1000\log_2 n$

C. $T(n) = n^2 - 1000\log_2 n$

D. $T(n) = 2n\log_2 n - 1000\log_2 n$

14. 下列函数中渐近时间复杂度最小的是 (A)。

A. $T(n) = \log_2 n + 5000n$

B. $T(n) = n^2 - 8000n$

C. $T(n) = n^3 + 5000n$

D. $T(n) = 2n\log_2 n - 1000n$

算法分析概论

15. 某算法的时间复杂度为 $O(n^2)$ ，表明该算法的 (C)。

A. 问题的规模是 n^2 B. 执行时间等于 n^2

C. 执行时间与 n^2 成正比 D. 问题规模与 n^2 成正比

16. 当输入非法错误时，一个“好”的算法会进行适当处理，不会产生难以理解的输出结果，这称为算法的 (B)。

A. 可读性 B. 健壮性 C. 正确性 D. 有穷性

14:35

Wednesday, October 16

26°C Good 27°C 75%

第二章：算法的性能分析

算法设计与分析



主讲教师：胡一涛